GLOBAL ACADEMY OF TECHNOLOGY

An Autonomous Institute, Affiliated to VTU Belagavi,
Approved by AICTE, Accredited by NAAC,'A' Grade,
Ideal Homes Township, Rajarajeshwari Nagar, Bengaluru – 560 098
Tel: +91 80 28603158 (Ext - 300), Email: hodcse@gat.ac.in Web: www.gat.ac.in

Department of Computer Science and Engineering

Computer Networks Laboratory Manual

V Semester
Course Code: BCS24502

Version 1.0
w.e.f 24.08.2026

Editorial Committee
CN Laboratory Faculty,
Dept. of CSE

Approved by
Dr. R Rajkumar
Professor & Head,
Department of CSE

DOCUMENT LOG

Name of the document	Machine Learning Laboratory Manual
Syllabus Scheme	2024
Current version number and date	1 / 24.08.2026
Subject code	BCS24502
Editorial Committee	CN Laboratory Faculty
Approved by	HOD, Dept. of CSE
Lab Faculty	Dr. Nagaveni V Ms. Yashaswini K Ms. Thaseen Taj Ms. Divya S J Mr. Praveen Kumar R
Computer Programmer	Dhanushree B V
Name of the document	Computer Networks
Date of Updation	24/8/2026
Authored by	Computer Networks Faculty
Approved by	HOD, CSE

Table of Contents

Sl. No.	Particulars.		Page No.
1	**Vision and Mission of The Department**		1
2	**PEOs, PSOs, POs**		2
3	**Course Details** • Course Objectives		4
4	**Syllabus** • Course Outcomes • Conduction of Practical Examination • CO-PO-PSO Mapping		5
5	**Tutorial Evaluation Process**		7
6	**CHAPTER 2**	**Programs**	
	1	Using Cisco Packet Tracer, create a topology and simulate sending a simple PDU from source to destination using hub and switch as connecting devices and demonstrate ping.	8
	2	Develop a C program to demonstrate Address Resolution Protocol (ARP).	17
	3	Develop a C program to compute: • CRC at the transmitter. • Syndrome at the receiver.	19
	4	Perform subnetting on IPv4 addresses using Cisco Packet Tracer.	22
	5	Develop a C program to demonstrate Link State Routing Algorithm.	27
	6	Develop a C program to demonstrate Distance Vector Routing Algorithm.	30
	7	Perform File Transfer in Client & Server using TCP/IP based Socket Programming.	33
	8	Use Wireshark to capture and analyze HTTP GET and POST requests.	36
	9	Develop a DNS Client-Server application and analyze DNS packets using Wireshark.	38
	Viva Questions		41

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision of the Department

To achieve academic excellence and strengthen the skills to meet emerging challenges of Computer Science and Engineering.

Mission of the Department

M1: To impart strong theoretical foundations in the field of Computer Science and Engineering accompanied with extensive practical skills.

M2: To inculcate research and innovation spirit through interaction with industry and carry out projects that address societal needs.

M3: Instill professional ethics and values with concern for environment.

Program Educational Objectives (PEOs) of Department

After the course completion, CSE graduates will be able to:

- Succeed in engineering/management positions with professional ethics.
- Engage in improving professional knowledge through certificate/post-graduate programs in engineering or management.
- Establish themselves as entrepreneurs and contribute to the Society.

Program Specific Outcomes (PSOs)

PSO1: Design, implement and test System Software and Application Software to meet the desired needs.

PSO2: Develop solutions in the area of Communication Networks, Database Systems and Computing Systems.

Program Outcomes (POs)

Engineering Graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6).

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9).

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Course Details

Course Name: Computer Networks
Course Code: BCS24502
Course prerequisite: C-Programming

Course Objectives

Upon completion of this course, students are expected to:

- Provide the basic knowledge of Computer Networks and its Models.
- To gain knowledge of the functionalities of the OSI/TCP/IP model.
- To upskill the knowledge of implementation of the 7 layers functionality such as flow control, error control and addressing schemes.
- To study the various protocols of all the layers and its applications.

SYLLABUS

Subject Code	BCS24502	CIE Marks	50
Hours/Week (L: T: P)	0:0:2	SEE Marks	50
Credits	04	Examination Hours	03

1	Using Cisco Packet Tracer, create a topology and simulate sending a simple PDU from source to destination using hub and switch as connecting devices and demonstrate ping.
2	Develop a C program to demonstrate Address Resolution Protocol (ARP).
3	Develop a C program to compute: • CRC at the transmitter. • Syndrome at the receiver.
4	Perform subnetting on IPv4 addresses using Cisco Packet Tracer.
5	Develop a C program to demonstrate Link State Routing Algorithm.
6	Develop a C program to demonstrate Distance Vector Routing Algorithm.
7	Perform File Transfer in Client & Server using TCP/IP based Socket Programming.
8	Use Wireshark to capture and analyse HTTP GET and POST requests.
9	Develop a DNS Client-Server application and analyse DNS packets using Wireshark.

Course Outcomes

The students should be able to:

CO1	Understand the concepts, architecture, and functionalities of computer networks and layered communication models.
CO2	Apply fundamental data communication techniques to support reliable and efficient network operations for Data link and network functionalities
CO3	Analyze addressing mechanisms, routing strategies, and network operations to facilitate effective connectivity.
CO4	Evaluate transport and application layer communication mechanisms for improved network performance and usability.

CO-PO-PSO MAPPING

CO/PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1	2	2	-	-	2	-	-	-	-	-	-	-	2
CO2	2	2	2	-	2	-	-	-	-	-	-	-	2
CO3	2	2	2	-	2	-	-	-	-	-	-	-	2
CO4	2	2	-	-	2	-	-	-	-	-	-	-	2

CIE Evaluation

Sl. No	Activity	Marks
1	Write-up	4
2	Execution	8
3	Viva Voce	3
	Total	**15**

LAB RUBRICS

Rubrics for Evaluation of Internal Test

Attribute	Max Marks	Good	Satisfactory	Poor
Write-up	**04**	**3–4 Marks** • Complete write-up. • Correct procedure and configuration/code. • Network diagram is correct. • Results are recorded properly.	**2–3 Marks** • Mostly complete write-up. • Minor errors in procedure/configuration. • Results are partially recorded.	**1 Mark** • Incomplete write-up. • Major errors in procedure/configuration. • Results are missing.
Execution	**08**	**6–8 Marks** • Performs the experiment correctly. • Correctly configures the network. • Gets the expected output. • Able to identify and correct errors.	**3–5 Marks** • Performs the experiment with some assistance. • Configuration is partially correct. • Gets the output with minor errors. • Needs help to correct errors.	**2 Marks** • Unable to perform the experiment. • Network configuration is incorrect. • Unable to correct errors. • Output is not obtained.
Viva Voce	**03**	**3 Marks** • Answers most questions correctly. • Explains the experiment clearly.	**2 Marks** • Answers basic questions. • Explains some parts of the experiment.	**1 Mark** • Unable to answer basic questions. • Unable to explain the experiment.
TOTAL	**15**			

1.Using Cisco Packet Tracer, create a topology and simulate sending a simple PDU from source to destination using hub and switch as connecting devices and demonstrate ping.

Theory

Cisco Packet Tracer is a network simulation and visualization tool developed by Cisco. It allows users to create, configure, and test computer networks without using physical networking devices.

In this experiment, a network topology is created using **PCs, a hub, and a switch**. A **Simple PDU** is used to simulate communication between a source and destination device. The **ping** command is used to verify connectivity between the devices.

A **Hub** operates at the **Physical Layer (Layer 1)** and broadcasts the received data to all connected devices.

A **Switch** operates mainly at the **Data Link Layer (Layer 2)** and forwards frames to the appropriate destination using **MAC addresses**.

Ping uses the **ICMP (Internet Control Message Protocol)** to test whether two network devices can reach each other.

Simple PDU in Packet Tracer provides a simple way to test packet delivery between network devices and observe whether the communication is successful.

We can create different network topologies using Cisco Packet Tracer. Let us explore different network diagrams starting with simple to bit complex.

End devices



Fig01:PC



Fig02:Server

Connections



Fig03:Copper Cross-Over



Fig04:Copper Stright-Through

Network Devices



Fig05:Router



Fig06:Switch



Fig07:Hub

Network Topology

1.This is a simple network topology with two networking devices connected via ethernet cable.Use an Automatic connecting cable to connect the devices with others.

Configure the PCs (hosts) and server with IPv4 address and Subnet Mask according to the IP addressing table above.

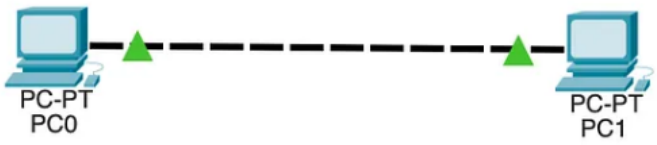


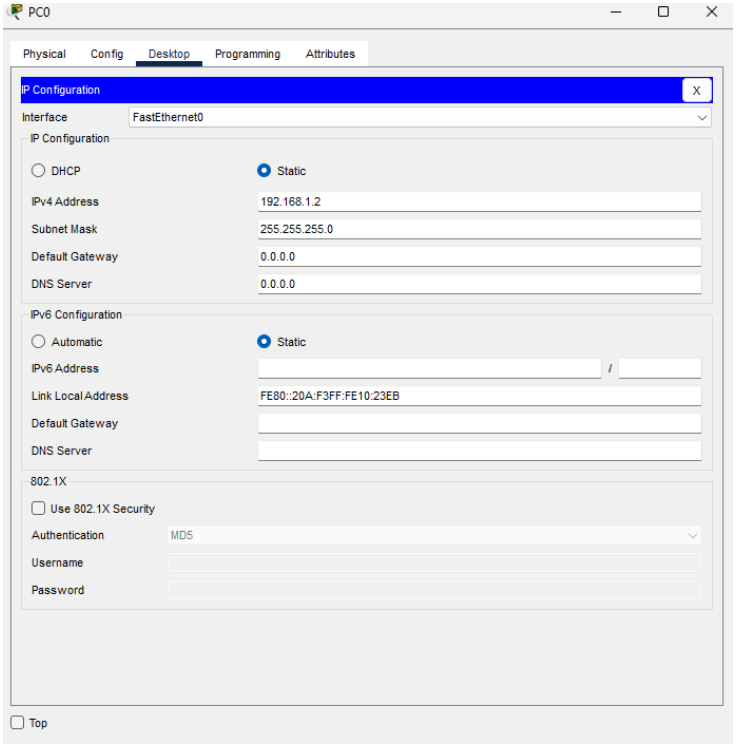
Fig08:Two PC-wired-network

To assign an IP address in PC0, click on PC0.

Then, go to desktop and IP configuration and there you will find IPv4 configuration.

Fill IPv4 address and subnet mask.

Similarly, repeat the same procedure with PC1 and server0 to configure them thoroughly.

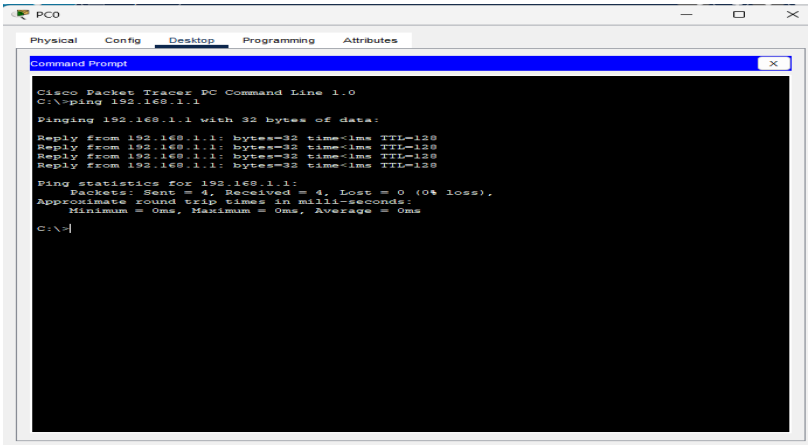


Verify the connection by pinging the IP address of any host in PC0.

Use the ping command to verify the connection.

We will check if we are getting any replies or not.

Here we get replies from a targeted node on both PCs. So, the connection is verified.



This topology connected many networking devices via the Hub as an intermediate device. We can connect many devices using the Hub depending upon of port available in the Hub.

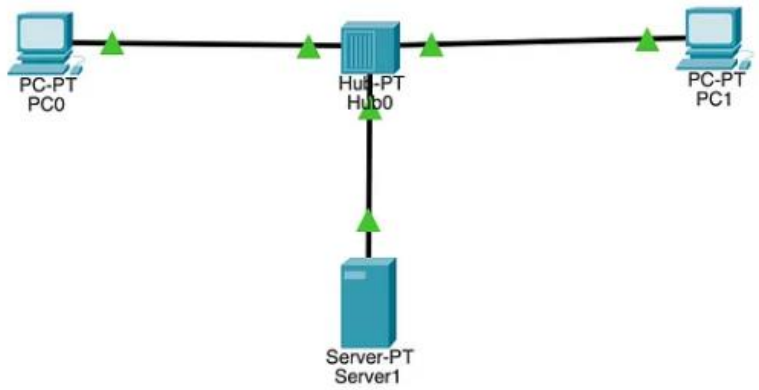


Fig09:Two PC and One server wired-network

Use an Automatic connecting cable to connect the devices with others.

Assign IP Addresses

Click on PC0 > Desktop > IP Configuration:

IP Address: 192.168.1.2

Subnet Mask: 255.255.255.0

Click on PC1 > Desktop > IP Configuration:

IP Address: 192.168.1.3

Subnet Mask: 255.255.255.0

Click on Server1 > Desktop > IP Configuration:

IP Address: 192.168.1.1

Subnet Mask: 255.255.255.0

Test the Network

Open Command Prompt in PC0:

Type ping 192.168.1.1 (to test connectivity to the Server).

Type ping 192.168.1.3 (to test connectivity to PC1).

This topology connected many networking devices via the Hub as an intermediate device. We can connect Hub To Hub as well in our network diagram.

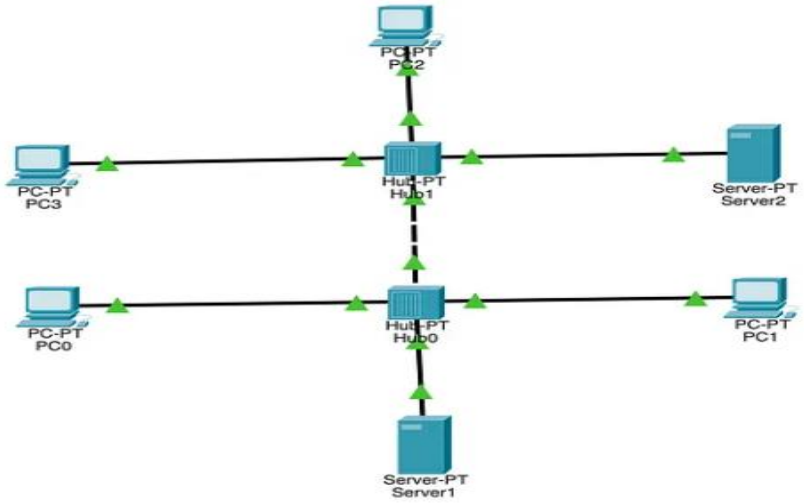


Fig10:Two Hub with many PCs and Server via wired-network

Assign IP Addresses

Device	IP Address	Subnet Mask
PC0	192.168.1.2	255.255.255.0
PC1	192.168.1.3	255.255.255.0
PC2	192.168.1.4	255.255.255.0
PC3	192.168.1.5	255.255.255.0
Server1	192.168.1.10	255.255.255.0
Server2	192.168.1.11	255.255.255.0

Test the Network

Open Command Prompt in PC0:

Type ping 192.168.1.1 (to test connectivity to the Server).

Type ping 192.168.1.3 (to test connectivity to PC1).

This topology connected many networking devices using the Router.



Fig11:Router with Two PCs via wired-network

Add Devices

Add 2 Servers:

From End Devices, drag two Server-PT (Server0 and Server1).

Add a Router:

From Network Devices > Routers.

Add Connections

Use Copper Straight-Through cables:

Connect Server0 → Router0 (Gig0/0)

Connect Server1 → Router0 (Gig0/1)

Configure IP Addresses

Let's assume the router interfaces and servers are on different subnets:

Device	Interface	IP Address	Subnet Mask
Server0	FA/0	192.168.1.2	255.255.255.0
Server1	FA/0	192.168.2.2	255.255.255.0
Router0	G0/0	192.168.1.1	255.255.255.0
Router0	G0/1	192.168.2.1	255.255.255.0

Configure Server IP Settings

Click each Server:

Desktop > IP Configuration

Server0:

IP Address: 192.168.1.2

Subnet Mask: 255.255.255.0

Default Gateway: 192.168.1.1

Server1:

IP Address: 192.168.2.2

Subnet Mask: 255.255.255.0

Default Gateway: 192.168.2.1

Configure Router Interfaces

Click Router0 > CLI tab: Router> enable

Router# configure terminal

Router(config)# interface gig0/0

Router(config-if)# ip address 192.168.1.1 255.255.255.0

Router(config-if)# no shutdown

Router(config-if)# exit

Router(config)# interface gig0/1

Router(config-if)# ip address 192.168.2.1 255.255.255.0

Router(config-if)# no shutdown

Router(config-if)# exit

Router(config)# exit

Router#

Test Connectivity

From Server0:

Open Command Prompt

Type ping 192.168.2.2 → to test connection to Server1

From Server1:

Ping back to 192.168.1.2

Both pings should succeed if everything is configured correctly.

5.This topology connected many networking devices using Switches and Routers.

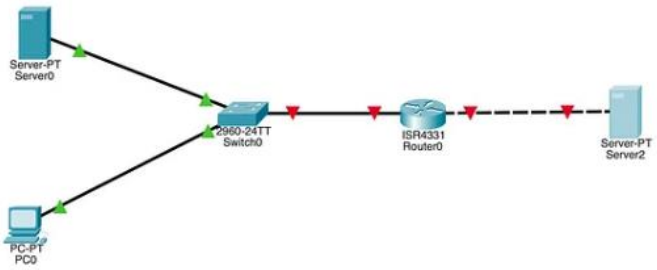


Fig12: Router, Switch with PC and Server via wired-network

Assign IP Addresses

At Server 0

IP: 192.168.1.2

Subnet Mask: 255.255.255.0

Default Gateway: 192.168.1.1

At PC0

IP: 192.168.1.3

Subnet Mask: 255.255.255.0

Default Gateway: 192.168.1.1

At Router0 (interface towards Switch0 - G0/0)

IP: 192.168.1.1

Subnet Mask: 255.255.255.0

At Router0 (interface towards Server2 - G0/1):

IP: 192.168.2.1

Subnet Mask: 255.255.255.0

At Server2

IP: 192.168.2.2

Subnet Mask: 255.255.255.0

Default Gateway: 192.168.2.1

Configure Router Interfaces

Click on Router0 → Go to CLI tab.

Enter the following commands:

enable

configure terminal

interface gig0/0

ip address 192.168.1.1 255.255.255.0

no shutdown

exit

interface gig0/1

ip address 192.168.2.1 255.255.255.0

no shutdown

exit

end

write

Test Connectivity

Click on PC0 → Desktop → Command Prompt.

Type:

ping 192.168.1.2 (ping Server0)

ping 192.168.2.2 (ping Server2 via router)

You should get replies if the configuration is correct.

2. Develop a C program to demonstrate Address Resolution Protocol (ARP).

```
#include <stdio.h>
#include <string.h>
int main()
{
   char ip[4][20] = {"192.168.1.1", "192.168.1.2",
                     "192.168.1.3", "192.168.1.4"};

   char mac[4][20] = {"AA:BB:CC:DD:EE:01",
                      "AA:BB:CC:DD:EE:02",
                      "AA:BB:CC:DD:EE:03",
                      "AA:BB:CC:DD:EE:04"};

   char searchIP[20];
   int i, found = 0;

   printf("Enter IP Address: ");
   scanf("%s", searchIP);

   for(i = 0; i < 4; i++)
   {
      if(strcmp(searchIP, ip[i]) == 0)
      {
         printf("\nARP Reply\n");
         printf("IP Address  : %s\n", ip[i]);
         printf("MAC Address : %s\n", mac[i]);
         found = 1;
         break;
      }
   }
   if(found == 0)
      printf("\nARP: IP Address not found in ARP Table.\n");
```

```
    return 0;
}
```

OUTPUT

```
Enter IP Address: 192.168.1.2
IP Address  : 192.168.1.2
MAC Address : AA:BB:CC:DD:EE:02
Enter IP Address: 192.168.1.10
ARP: IP Address not found in ARP Table.
```

3. Develop a C program to compute:

- **CRC at the transmitter.**
- **Syndrome at the receiver.**

```
#include <stdio.h>
#include <string.h>

void crc(char data[], char key[])
{
   int i, j, n = strlen(data), k = strlen(key);

   for (i = 0; i <= n - k; i++)
   {
      if (data[i] == '1')
      {
         for (j = 0; j < k; j++)
            data[i + j] = (data[i + j] == key[j]) ? '0' : '1';
      }
   }
}

int main()
{
   char data[50], key[20];
   char temp[100], crc_value[20], codeword[100];

   int n, k, i;

   printf("Enter data: ");
   scanf("%s", data);

   printf("Enter generator: ");
   scanf("%s", key);
```

```
    n = strlen(data);
    k = strlen(key);

    /* Transmitter */
    strcpy(temp, data);

    for (i = 0; i < k - 1; i++)
       strcat(temp, "0");

    crc(temp, key);

    /* Store CRC */
    strcpy(crc_value, temp + n);

    printf("\nCRC = %s", crc_value);

    /* Create codeword */
    strcpy(codeword, data);
    strcat(codeword, crc_value);

    printf("\nCodeword = %s", codeword);

    /* Receiver */
    crc(codeword, key);

    printf("\nSyndrome = %s", codeword + n);

    return 0;
}
```

Input:

Enter data: 1101011011

Enter generator: 10011

Output:

CRC = 1110

Codeword = 11010110111110

Syndrome = 0000

4. Perform subnetting on IPv4 addresses using Cisco Packet Tracer.

A subnet, or subnetwork, is a part of a larger network. Subnets are a logical part of an IP network into multiple, smaller network components. The Internet Protocol (IP) is the method for transmitting data from one computer to another over the internet network. Each computer, or host, on the internet, has at least one IP address as a unique identifier

Steps to Configure and Verify Three Router Connections in Cisco Packet Tracer:

Step 1: First, open the Cisco packet tracer desktop and select the devices given below:

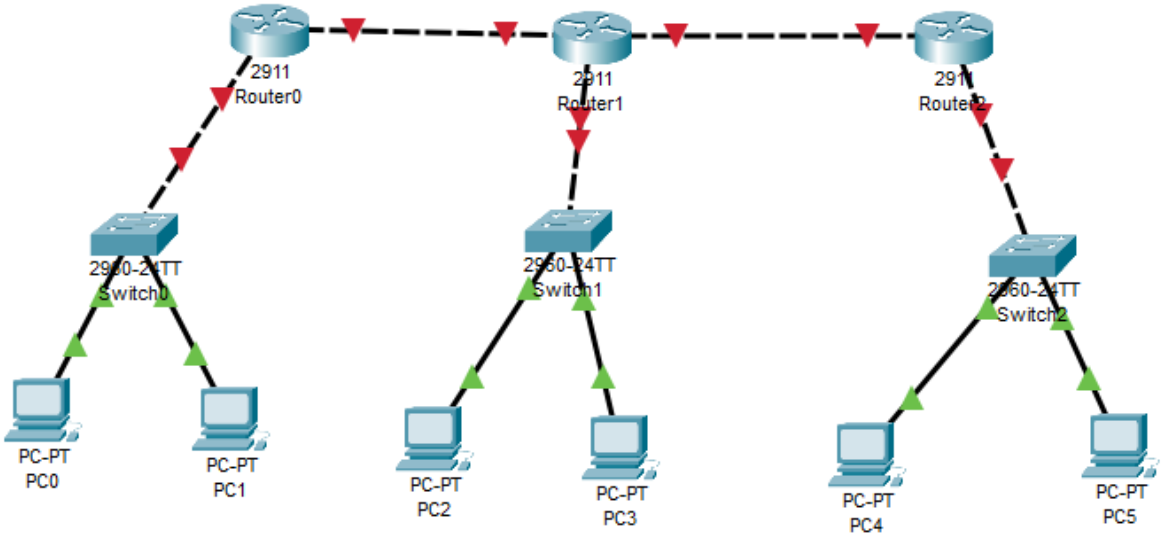
Sl.No	Device	Model-name	Qty
1	PC	PC	6
2	Switch	PT-Switch	3
3	Router	PT-Router	3

IP Addressing Table for PCs

S.NO	Device	IPv4 Address	Subnet Mask	Default-Gateway
1.	pc0	192.168.1.2	255.255.255.0	192.168.1.1
2.	pc1	192.168.1.3	255.255.255.0	192.168.1.1
3.	pc2	192.168.2.2	255.255.255.0	192.168.2.1
4.	pc3	192.168.2.3	255.255.255.0	192.168.2.1
5.	pc4	192.168.3.2	255.255.255.0	192.168.3.1
6.	pc5	192.168.3.3	255.255.255.0	192.168.3.1

Then, create a network topology as shown below the image.

Use an Automatic connecting cable to connect the devices with others.

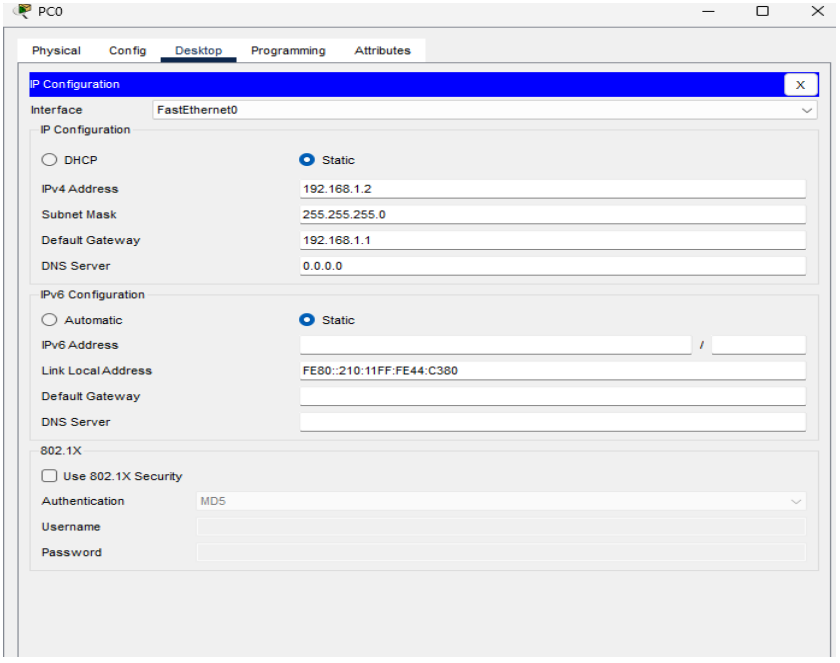


Step 2: Configure the PCs (hosts) with IPv4 address and Subnet Mask according to the IP addressing table given above.

To assign an IP address in PC0, click on PC0.

Then, go to desktop and then IP configuration and there you will IPv4 configuration.

Fill IPv4 address and subnet mask.



Repeat the same procedure with other PCs to configure them thoroughly.

Step 3: Configure router with IP address and subnet mask.

IP Addressing Table Router

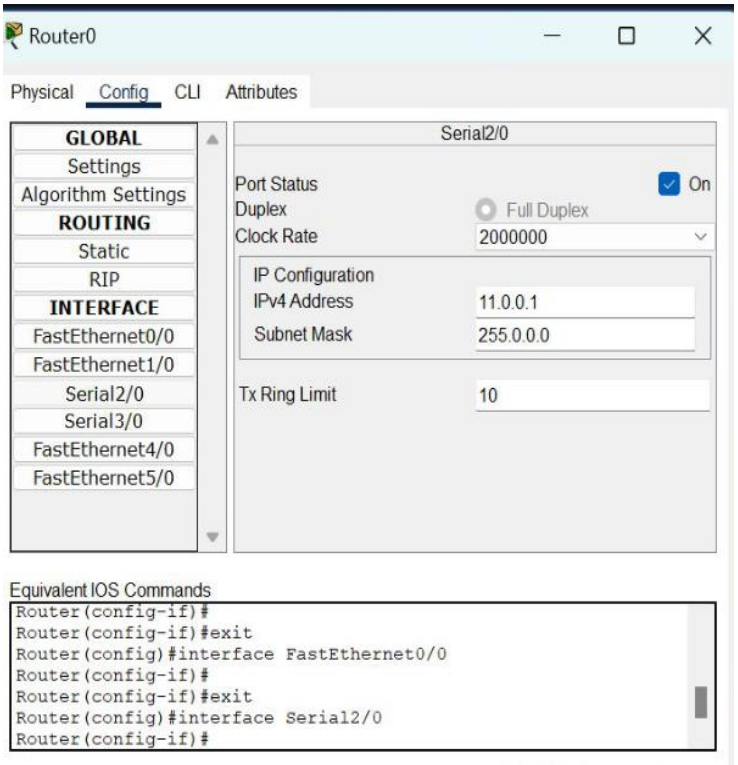
NO	Device	Interface	IPv4 Address	Subnet mask
1.	router0	FastEthernet0/0	192.168.1.1	255.255.255.0
		Serial2/0	11.0.0.1	255.0.0.0
2.	router1	Serial 2/0	11.0.0.2	255.0.0.0
		Serial 3/0	12.0.0.1	255.0.0.0
3.	router 3	FastEthernet0/0	192.168.3.1	255.255.255.0
		Serial2/0	12.0.0.2	255.0.0.0

To assign an IP address in router0, click on router0.

Then, go to config and then Interfaces.

Then, configure the IP address in FastEthernet and serial ports according to IP addressing Table.

Fill IPv4 address and subnet mask.



Repeat the same procedure with other routers to configure them thoroughly.

Step 4: After configuring all of the devices we need to assign the routes to the routers.

To assign static routes to the particular router:

First, click on router0 then Go to CLI.

Then type the commands and IP information given below.

CLI command : ip route <network id> <subnet mask><next hop>

Static Routes for Router0 are given below:

Router(config)#ip route 192.168.2.0 255.255.255.0 11.0.0.2

Router(config)#ip route 11.0.0.0 255.0.0.0 11.0.0.2

Router(config)#ip route 192.168.3.0 255.255.255.0 11.0.0.2

Router(config)#ip route 12.0.0.0 255.0.0.0 11.0.0.2

Static Routes for Router1 are given below:

Router(config)#ip route 192.168.1.0 255.255.255.0 11.0.0.1

Router(config)#ip route 11.0.0.0 255.0.0.0 11.0.0.1

Router(config)#ip route 192.168.3.0 255.255.255.0 12.0.0.2

Router(config)#ip route 12.0.0.0 255.0.0.0 12.0.0.2

Static Routes for Router2 are given below:

Router(config)#ip route 192.168.1.0 255.255.255.0 12.0.0.1

Router(config)#ip route 11.0.0.0 255.0.0.0 12.0.0.1

Router(config)#ip route 12.0.0.0 255.0.0.0 12.0.0.1

Router(config)#ip route 192.168.2.0 255.255.255.0 12.0.0.1

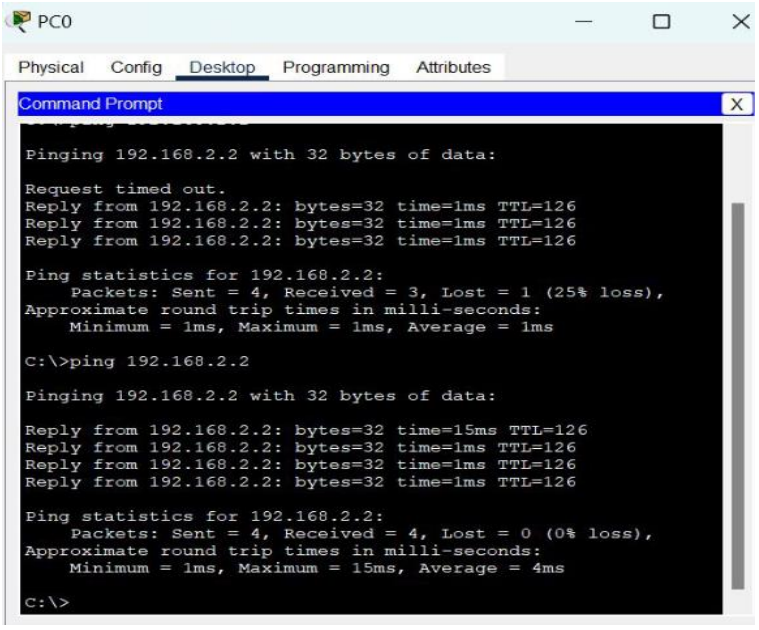
Step 5: Verifying the network by pinging the IP address of any PC. We will use the ping command to do so.

First, click on PC0 then Go to the command prompt

Then type ping <IP address of targeted node>

As we can see in the below image we are getting replies which means the connection is working very fine

Example : ping 192.168.2.2

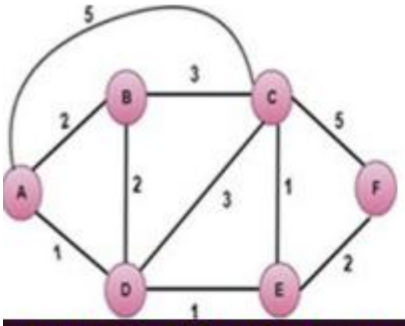


5. Implement Link-State and Distance Vector routing protocols.

```
#include<stdio.h>
#include<string.h>
#include<math.h>
int main()
{
int u,v,num,i,j,l,k,s[10],min,cost[10][10],dist[10],path[10],n;
printf("\n ENTER VERTECES:");
scanf("%d",&n);
printf("\n ENTER ADJECENCY MATRIX:\n");
for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
scanf("%d",&cost[i][j]);
}
for(i=1;i<=n;i++)
for(j=1;j<=n;j++)
if(i==j) cost[i][j]=0;
else if(cost[i][j]==-1)
cost[i][j]=30000;
printf("\nENTER SOURCE VERTEX:");
scanf("%d",&v);
for(i=1;i<=n;i++)
{
s[i]=0;
path[i]=v; dist[i]=cost[v][i];
}
dist[v]=0;
for(num=2;num<=n;num++)
{
```

```
min=30000; u=0;
for(i=1;i<=n;i++)
{
if(s[i]!=1) if(min>dist[i])
{
u=i; min=dist[i];
}
}
s[u]=1;
for(i=1;i<=n;i++)
{
if(s[i]!=1)
if(dist[i]>(min+cost[u][i]))
{
dist[i]=min+cost[u][i];
path[i]=u;
}
}
}
printf("\n");
printf("\nPATH MATRIX:\n");
printf("\nDISTANCE NODE PATH\n");
for(i=1;i<=n;i++)
{
printf("\n %d",dist[i]);
printf(" %d ",i);
j=i;
do
{
printf(" --> %d ",path[j]);
```

```
u=path[j];
j=u;
}while(u!=v);
}
return 0;
}
```




```
admin1@admin1-ThinkCentre-M70q-Gen-2:~/Desktop/CN_lab$ gedit link.c
admin1@admin1-ThinkCentre-M70q-Gen-2:~/Desktop/CN_lab$ cc link.c -o link
admin1@admin1-ThinkCentre-M70q-Gen-2:~/Desktop/CN_lab$ ./link

 ENTER VERTECES:6

 ENTER ADJECENCY MATRIX:
0 2 5 1 999 999
2 0 3 2 999 999
5 3 0 3 1 5
1 2 3 0 1 999
999 999 1 1 0 2
999 999 5 999 2 0

ENTER SOURCE VERTEX:1


PATH MATRIX:

DISTANCE   NODE PATH

 0 1   --> 1
 2 2   --> 1
 3 3   --> 5   --> 4   --> 1
 1 4   --> 1
 2 5   --> 4   --> 1
 4 6   --> 5   --> 4   --> 1 admin1@admin1-ThinkCentre-M70q-Gen-2:~/Desktop/
```

6.Write a program to demonstrate Distance vector routing algorithm

```
#include<stdio.h>
int dist[50][50],temp[50][50],n,i,j,k,x;
void dvr();
int main()
{
printf("\nEnter the number of nodes : ");
 scanf("%d",&n);
 printf("\nEnter the distance matrix :\n");
 for(i=0;i<n;i++)
 {
 for(j=0;j<n;j++)
 {
 scanf("%d",&dist[i][j]);
 dist[i][i]=0;
 temp[i][j]=j;
 }
 printf("\n");
}
          dvr();
          return 0;
}
void dvr()
{
for (i = 0; i < n; i++)
 for (j = 0; j < n; j++)
                    for (k = 0; k < n; k++)
 if (dist[i][k] + dist[k][j] < dist[i][j])
                              {
 dist[i][j] = dist[i][k] + dist[k][j];
 temp[i][j] = k;
 }
for(i=0;i<n;i++)
```

```
{
printf("\n\nState value for router %d is \n",i+1);
for(j=0;j<n;j++)
        printf("\t\nnode %d via %d Distance%d",j+1,temp[i][j]+1,dist[i][j]);
}
printf("\n\n");
}
```

OUTPUT

```
Enter the number of nodes : 4
Enter the distance matrix :

0 2 7 1
2 0 1 5
7 1 0 3
1 5 3 0

State value for router 1 is
        node 1 via 1 Distance 0
        node 2 via 2 Distance 2
        node 3 via 2 Distance 3
        node 4 via 4 Distance 1

State value for router 2 is
        node 1 via 1 Distance 2
        node 2 via 2 Distance 0
        node 3 via 3 Distance 1
        node 4 via 1 Distance 3

State value for router 3 is
        node 1 via 2 Distance 3
        node 2 via 2 Distance 1
        node 3 via 3 Distance 0
        node 4 via 4 Distance 3

State value for router 4 is

        node 1 via 1 Distance 1
        node 2 via 1 Distance 3
        node 3 via 3 Distance 3
        node 4 via 4 Distance 0
```

7.Perform File Transfer in Client & Server using TCP/IP based Socket Programming.

Server program

Save as server.c:

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main()
{
   int server_socket, client_socket;
   char buffer[1024];
   FILE *fp;

   struct sockaddr_in server, client;
   socklen_t len = sizeof(client);

   server_socket = socket(AF_INET, SOCK_STREAM, 0);

   server.sin_family = AF_INET;
   server.sin_port = htons(8080);
   server.sin_addr.s_addr = INADDR_ANY;

   bind(server_socket, (struct sockaddr *)&server, sizeof(server));

   listen(server_socket, 5);

   printf("Server waiting for client...\n");


client_socket = accept(server_socket,
                       (struct sockaddr *)&client,
                       &len);

   printf("Client connected.\n");

   fp = fopen("received.txt", "w");

   while (1)
   {
      int n = recv(client_socket, buffer, sizeof(buffer), 0);

      if (n <= 0)
         break;

      fwrite(buffer, 1, n, fp);
```

```
    }

    printf("File received successfully.\n");

    fclose(fp);
    close(client_socket);
    close(server_socket);

    return 0;
}
```

Client program

Save as **client.c**

```
#include <stdio.h>
#include <unistd.h>
#include <arpa/inet.h>

int main()
{
    int sock;
    char buffer[1024];
    FILE *fp;

    struct sockaddr_in server;
    sock = socket(AF_INET, SOCK_STREAM, 0);

    server.sin_family = AF_INET;
    server.sin_port = htons(8080);
    server.sin_addr.s_addr = inet_addr("127.0.0.1");

    connect(sock, (struct sockaddr *)&server, sizeof(server));

    printf("Connected to server.\n");

    fp = fopen("sample.txt", "r");

    while (1)
    {
        int n = fread(buffer, 1, sizeof(buffer), fp);

        if (n <= 0)
            break;
        send(sock, buffer, n, 0);
    }

    printf("File sent successfully.\n");

    fclose(fp);
    close(sock);
    return 0;
}
```

Procedure for Execution

1. Create a folder named **TCP_File_Transfer**.
2. Create the following three files inside the folder:
   - `server.c`
   - `client.c`
   - `sample.txt`
3. Copy the server program into **`server.c`** and save it.
4. Copy the client program into **`client.c`** and save it.
5. Open **`sample.txt`** and enter some text, for example:

```
This is a sample file for TCP file transfer.
```

6. Open **Command Prompt/Terminal** and navigate to the folder containing the files.
7. Compile the server program:

```
gcc server.c -o server
```

8. Compile the client program:

```
gcc client.c -o client
```

9. Open the **first terminal** and execute the server:

```
./server
```

10. The server will display:

```
Server waiting for client...
```

11. Open a **second terminal** and navigate to the same folder.
12. Execute the client:

```
./client
```

13. The client connects to the server using **IP address** `127.0.0.1` and **port number** `8080`.
14. The client reads `sample.txt` and sends its contents to the server.
15. The server receives the file data and stores it in **`received.txt`**.
16. The client displays:

```
Connected to server.
File sent successfully.
```

17. The server displays:

```
Client connected.
File received successfully.
```

18. Open **`received.txt`** and verify that it contains the same data as **`sample.txt`**.

Output

Server Side:

```
Server waiting for client...
Client connected.
File received successfully.
```

Client Side:

```
Connected to server.
File sent successfully.
```

8. Use Wireshark to capture and analyze HTTP GET and POST requests.

1. The Basic HTTP GET/response interaction

Let's begin our exploration of HTTP by downloading a very simple HTML file - one that is very short, and contains no embedded objects. Do the following:

1. Start up your web browser.
2. Start up the Wireshark packet sniffer, as described in the Introductory lab (but don't yet begin packet capture). Enter "http" (just the letters, not the quotation marks, and in lower case) in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window. (We're only interested in the HTTP protocol here, and don't want to see the clutter of all captured packets).
3. Wait a bit more than one minute (we'll see why shortly), and then begin Wireshark packet capture.
4. Enter the following to your browser http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html
   Your browser should display the very simple, one-line HTML file.
5. Stop Wireshark packet capture.

Your Wireshark window should look similar to the window shown in Figure 1. If you're unable to run Wireshark on a live network connection, you can download a packet trace that was created when the steps above were followed.[1]

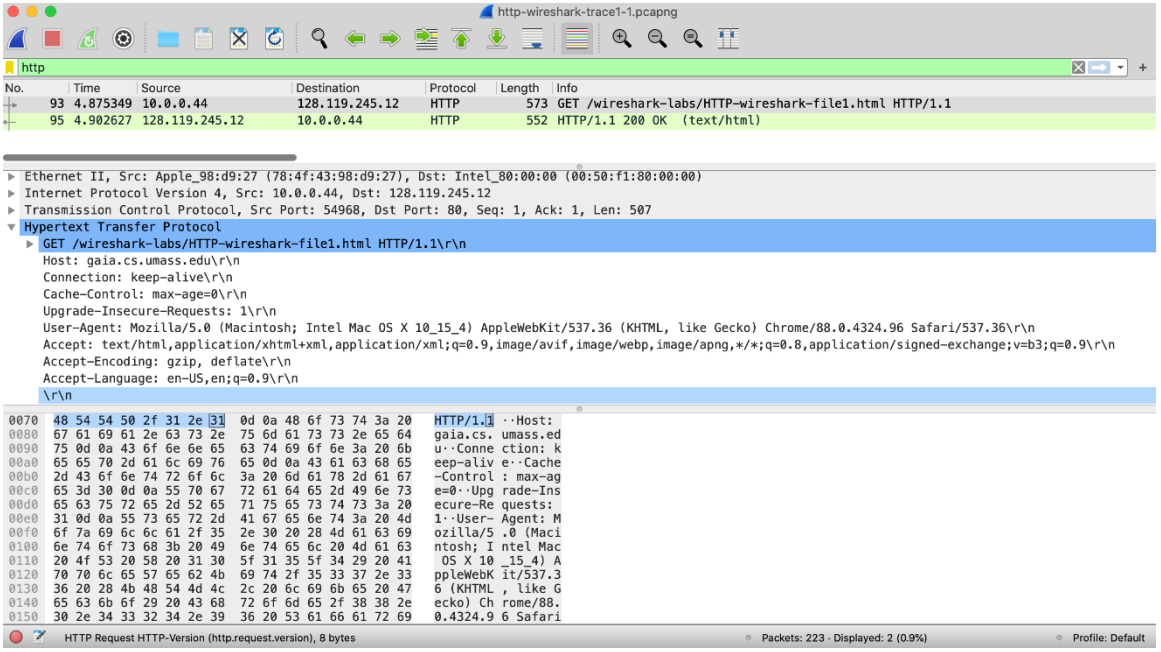


Figure 1: Wireshark Display after http://gaia.cs.umass.edu/wireshark-labs/ HTTP-wireshark-file1.html has been retrieved by your browser

The example in Figure 1 shows in the packet-listing window that two HTTP messages were captured:

the GET message (from your browser to the gaia.cs.umass.edu web server) and the response message from the server to your browser. The packet-contents window shows details of the selected message (in this case the HTTP OK message, which is highlighted in the packet-listing window). Recall that since the HTTP message was carried inside a TCP segment, which was carried inside an IP datagram, which was carried within an Ethernet frame, Wireshark displays the Frame, Ethernet, IP, and TCP packet information as well. We want to minimize the amount of non-HTTP data displayed (we're interested in HTTP here, and will be investigating these other protocols is later labs), so make sure the boxes at the far left of the Frame, Ethernet, IP and TCP information have a plus sign or a right-pointing triangle (which means there is hidden, undisplayed information), and the HTTP line has a minus sign or a down-pointing triangle (which means that all information about the HTTP message is displayed).

By looking at the information in the HTTP GET and response messages, answer the following questions. If you're doing this lab as part of class, your teacher will provide details about how to hand in assignments, whether written or in an LMS.[2]

1. Is your browser running HTTP version 1.0, 1.1, or 2? What version of HTTP is the server running?
2. What languages (if any) does your browser indicate that it can accept to the server?
3. What is the IP address of your computer? What is the IP address of the gaia.cs.umass.edu server?
4. What is the status code returned from the server to your browser?
5. When was the HTML file that you are retrieving last modified at the server?
6. How many bytes of content are being returned to your browser?
7. By inspecting the raw data in the packet content window, do you see any headers within the data that are not displayed in the packet-listing window? If so, name one.

In your answer to question 5 above (assuming you're running Wireshark "live", as opposed to using an earlier-recorded trace file), you might have been surprised to find that the document you just retrieved was last modified within a minute before you downloaded the document. That's because (for this particular file), the gaia.cs.umass.edu server is setting the file's last-modified time to be the current time, and is doing so once per minute. Thus, if you wait a minute between accesses, the file will appear to have been recently modified, and hence your browser will download a "new" copy of the document.

9.Develop a DNS Client-Server application and analyse DNS packets using Wireshark.

Aim

To demonstrate DNS client-server communication using `nslookup` and analyse DNS query and response packets using Wireshark.

Software Required

- Wireshark
- Command Prompt / Terminal
- Internet connection

Theory

The **Domain Name System (DNS)** translates hostnames into IP addresses. A DNS client sends a DNS query to a DNS server and receives a DNS response containing the required information. The client generally communicates with its local DNS server, while the DNS server may contact other DNS servers to resolve the requested hostname.

The `nslookup` command can be used to query a DNS server for DNS records. For example, when the user enters a hostname, `nslookup` sends a DNS query and displays the response received from the DNS server.

Procedure

Part A: Generate DNS Traffic

1. Open **Command Prompt**.
2. Open **Wireshark**.
3. Select the active network interface such as **Wi-Fi** or **Ethernet**.
4. Start packet capture in Wireshark.
5. Open Command Prompt and execute:

```
nslookup www.google.com
```

6. The DNS client sends a query to the DNS server and receives a response.
7. Stop the Wireshark packet capture.

The original Wireshark exercise uses `nslookup` to generate DNS queries and then captures the resulting DNS messages.

Part B: Filter DNS Packets

In the Wireshark display filter box, enter:

```
dns
```

Press **Enter**.

Wireshark will display the DNS packets captured during the `nslookup` operation.

Part C: Analyse DNS Query Packet

Select the DNS **Query** packet and expand the following fields:

Internet Protocol → User Datagram Protocol → Domain Name System

Observe the following parameters:

- Source IP address
- Destination IP address
- Source port
- Destination port
- DNS transaction ID
- Query name
- Query type
- Number of questions
- Number of answers

The DNS lab specifically requires analysis of the destination port, source port, destination IP address, query type, questions, and answers.

Part D: Analyse DNS Response Packet

Select the corresponding DNS **Response** packet.

Observe:

- Source IP address
- Destination IP address
- Source port
- Destination port
- DNS transaction ID
- Number of questions
- Number of answers
- IP address returned for the requested hostname

The DNS response can be examined to determine the number of questions and answers returned by the DNS server.

Observation Table

Parameter	DNS Query	DNS Response
Source IP	_____	_____
Destination IP	_____	_____
Source Port	_____	_____
Destination Port	_____	_____
Protocol	_____	_____
Query Name	_____	_____
Query Type	_____	_____
Number of Questions	_____	_____
Number of Answers	_____	_____

Viva Questions

1. What is Cisco Packet Tracer?

   Cisco Packet Tracer is a network simulation tool used to create, configure, and test computer networks virtually.

2. What is a hub?

   A hub is a Layer 1 device that broadcasts incoming data to all connected devices.

3. What is a switch?

   A switch is a Layer 2 device that forwards frames to the specific destination device using MAC addresses.

4. What is PDU in Packet Tracer?

   PDU stands for Protocol Data Unit. In Packet Tracer, a Simple PDU can be used to simulate communication between two devices.

5. What is ARP?

   ARP stands for Address Resolution Protocol. It is used to find the MAC address corresponding to an IPv4 address.

6. Why is ARP required?

   ARP is required because communication within a local network uses MAC addresses, while applications generally identify destinations using IP addresses.

7. What does an ARP Request contain?

   It contains the IP address whose corresponding MAC address needs to be found.

8. What does an ARP Reply contain?

   It provides the MAC address associated with the requested IP address.

9. Where is the ARP information stored?

   ARP information is stored temporarily in the ARP cache/table.

10. What is CRC?

    CRC stands for Cyclic Redundancy Check. It is an error-detection technique used to detect errors in transmitted data.

11. Where is CRC calculated?

    CRC is calculated at the transmitter before sending the data.

12. What is a generator polynomial?

    It is the predefined binary sequence used as the divisor during CRC calculation.

13. What is syndrome in CRC?

    The syndrome is the remainder obtained after dividing the received data by the generator polynomial.

14. What is subnetting?

    Subnetting is the process of dividing a large IP network into smaller logical networks called subnets.

15. Why is subnetting used?

    It improves IP address utilization, reduces broadcast traffic, and helps organize networks.

16. What is a subnet mask?

    A subnet mask identifies which portion of an IPv4 address represents the network and which portion represents the host.

17. What is CIDR?

    CIDR stands for Classless Inter-Domain Routing. It represents an IP network using prefix notation such as /24.

18. How do you test connectivity between two devices in Packet Tracer?

    We can use the ping command or Simple PDU to test connectivity.

19. What is Link State Routing?

    Link State Routing is a routing technique in which routers maintain information about the network topology and calculate the shortest path.

20. Which algorithm is commonly used in Link State Routing?

    Dijkstra's algorithm is commonly used.

21. What information does a router maintain?

    It maintains information about its neighboring routers and the cost of links.

22. What is the purpose of Dijkstra's algorithm?

    It is used to find the shortest path from a source node to all other nodes.

23. Give an example of a Link State routing protocol.

    OSPF (Open Shortest Path First) is a Link State routing protocol.

24. What is Distance Vector Routing?

    It is a routing method in which routers exchange information about distances and paths to reach different networks.

25. Which algorithm is commonly associated with Distance Vector Routing?

    Bellman-Ford algorithm.

26. What information is maintained in a distance vector table?

It generally contains the destination, distance/cost, and next hop.

27. Give an example of a Distance Vector routing protocol.

RIP (Routing Information Protocol).

28. What is a major problem with Distance Vector Routing?

It can suffer from the count-to-infinity problem.

29. What is a socket?

A socket is an endpoint used for communication between two devices over a network.

30. Which protocol is used in this experiment?

TCP (Transmission Control Protocol).

31. Why is TCP used for file transfer?

TCP provides reliable, ordered, and error-checked delivery of data.

32. What is the role of the server?

The server waits for a client connection, receives the file data, and stores it.

33. What is the role of the client?

The client establishes a connection with the server and sends the file data.

34. What is Wireshark?

Wireshark is a network packet analyzer used to capture and analyze network packets.

35. What is HTTP GET?

GET is an HTTP request method used to request a resource from a server.

36. What is HTTP POST?

POST is an HTTP request method used to send data to a server.

37. What Wireshark filter can be used to find GET and POST requests?

http.request.method == GET or http.request.method == POST

38. What is the purpose of Follow TCP Stream in Wireshark?

It allows us to view the complete TCP conversation between the client and server, including the HTTP request and response.

39. What is DNS?

DNS stands for Domain Name System. It translates domain names into IP addresses.

40. Why is DNS required?

DNS allows users to access websites using easy-to-remember domain names instead of IP addresses.

41. Which port is commonly used by DNS?

DNS commonly uses port 53.

42. Which transport protocol is normally used for DNS queries?

DNS normally uses UDP, although TCP is also used in certain situations.

43. How can DNS packets be filtered in Wireshark?

Use the display filter: